Introduction to Hash Tables

1. Introduction: Why Hash Tables?

The Search Problem

- **Unsorted Arrays/Lists:** $O(N)$ search time.
- **Sorted Arrays (Binary Search):** $O(\log N)$ search, but $O(N)$ insertion/deletion.
- **Balanced Binary Search Trees (e.g., `TreeMap`):** $O(\log N)$ for search, insertion, and deletion.

The Hash Table Solution

Hash tables provide $O(1)$ **average-case time complexity** for insertions, deletions, and lookups by mapping keys directly to array indices using a **hash function**.

Real-World Applications

- **Databases:** Indexing for fast record retrieval.
- **Compilers & Interpreters:** Symbol tables storing variable names and scopes.
- **Caches:** Web caching and memoization (e.g., LRU Cache).
- **Networking:** Routing tables and packet filtering.

---

2. Anatomy of a Hash Table & Collision Resolution

A hash table consists of an underlying array of a fixed size. A key is passed through a **hash function** to compute an index.

The Pigeonhole Principle & Collisions

Since the key space is typically larger than the array size, two different keys will occasionally map to the same index (**collision**).

Primary Collision Resolution Strategies

1. **Separate Chaining:** Each array index (bucket) points to a linked list (or balanced tree) of entries that hashed to that index. (Used by Java's `HashMap`).
2. **Open Addressing:** All elements are stored directly in the array. When a collision occurs, probe for the next available slot (Linear Probing, Quadratic Probing, Double Hashing).

---

3. What Makes a Good Hash Function?

A hash function $h(k)$ transforms a key k into an integer array index. For optimal performance, a good hash function must be:

1. **Deterministic:** The exact same key must *always* produce the exact same hash code.
2. **Efficient to Compute:** $O(1)$ time complexity so it doesn't bottleneck operations.

3. **Uniform Distribution:** It must spread keys evenly across the available buckets to minimize clustering and collisions.
4. **Depends on All Key Components:** If a key consists of multiple fields, all fields should influence the final hash value.

---

4. The Java `hashCode()` Function & Contract

In Java, every object inherits the `hashCode()` method from `Object`.

The Crucial Contract:

- If two objects are equal according to the `equals(Object)` method, they **must** return the exact same integer from `hashCode()`.
- If two objects have the same `hashCode()`, they are **not** required to be equal (this is a collision).

Java Example: Overriding `hashCode()` and `equals()`

When creating custom objects as keys in a hash structure, both methods must be overridden together.

```java
import java.util.Objects;

public class Student {
    private String studentId;
    private String email;

    public Student(String studentId, String email) {
        this.studentId = studentId;
        this.email = email;
    }

    // Good hashCode implementation using java.util.Objects
    @Override
    public int hashCode() {
        return Objects.hash(studentId, email);
    }

    // Must align with hashCode contract
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Student student = (Student) obj;
        return Objects.equals(studentId, student.studentId) &&
               Objects.equals(email, student.email);
    }
}
```

---

5. Implementation: Simplified Hash Table with Separate Chaining

This stripped-down implementation demonstrates how a hash table manages buckets, computes indices, and handles collisions via chaining.

```
import java.util.LinkedList;
import java.util.Objects;

public class SimpleHashTable<K, V> {
    private class Entry {
        K key;
        V value;

        Entry(K key, V value) {
            this.key = key;
            this.value = value;
        }
    }

    private static final int CAPACITY = 10;
    private LinkedList<Entry>[] buckets;

    @SuppressWarnings("unchecked")
    public SimpleHashTable() {
        buckets = new LinkedList[CAPACITY];
        for (int i = 0; i < CAPACITY; i++) {
            buckets[i] = new LinkedList<>();
        }
    }

    // Hash function mapping key to array index
    private int getBucketIndex(K key) {
        int hashCode = (key == null) ? 0 : key.hashCode();
        // Math.abs handles negative hash codes, modulo maps to capacity
bounds
        return Math.abs(hashCode) % CAPACITY;
    }

    public void put(K key, V value) {
        int index = getBucketIndex(key);
        LinkedList<Entry> bucket = buckets[index];

        // Check if key already exists, update if found
        for (Entry entry : bucket) {
            if (Objects.equals(entry.key, key)) {
                entry.value = value;
                return;
            }
        }

        // Otherwise, add new entry to bucket chain
        bucket.add(new Entry(key, value));
    }
```

```
    public V get(K key) {
        int index = getBucketIndex(key);
        LinkedList<Entry> bucket = buckets[index];

        for (Entry entry : bucket) {
            if (Objects.equals(entry.key, key)) {
                return entry.value;
            }
        }
        return null; // Key not found
    }
}
```

---

6. Detailed Collision Resolution Strategies

1. Separate Chaining

Each array index (bucket) points to a secondary data structure (a linked list or balanced BST).

- **Pros:** Easy to implement, handles load factors $\lambda > 1.0$ gracefully, simple deletion.
- **Cons:** Extra memory overhead for pointers, poor CPU cache locality due to scattered nodes.

---

2. Open Addressing: Linear Probing

Probes adjacent slots sequentially: $(i + 1) \pmod N, (i + 2) \pmod N, \dots$

- **Pros:** Excellent CPU cache locality, no pointer overhead.
- **Cons:** Vulnerable to **Primary Clustering**, where consecutive filled slots merge into massive contiguous blocks.

---

3. Open Addressing: Quadratic Probing

Uses a quadratic sequence for probing: $(h(\text{key}) + i^2) \pmod N$.

- **Pros:** Eliminates primary clustering by jumping across the array.
- **Cons:** Subject to **Secondary Clustering** (keys with the same initial hash follow the exact same probe sequence); probe sequence may fail to find an open slot if $\lambda \geq 0.5$.

---

4. Open Addressing: Double Hashing

Uses a secondary hash function $h_2(\text{key})$ to determine step size: $(h_1(\text{key}) + i \cdot h_2(\text{key})) \pmod N$.

- **Pros:** Virtually eliminates both primary and secondary clustering; yields uniform distribution.
- **Cons:** Computationally more expensive due to dual hash calculations; step size h_2 must never equal 0.

Collision Strategy Comparison Matrix

Strategy	Memory Efficiency	Cache Performance	Clustering Risk	Max Recommended Load Factor (λ)
Separate Chaining	Moderate (Pointer overhead)	Poor (Scattered nodes)	None	Can exceed 1.0
Linear Probing	High (Array only)	Excellent	High (Primary)	Must stay < 0.7
Quadratic Probing	High (Array only)	Good	Moderate (Secondary)	Must stay < 0.5
Double Hashing	High (Array only)	Good	Very Low	Must stay < 0.7

7. Summary & Key Takeaways

- **Speed vs. Ordering:** Hash tables offer $O(1)$ **average-case performance** for search, insertion, and deletion, but offer **no sorting/ordering guarantee**.
- **Quality of Hash Functions:** A poor hash function degrades performance down to $O(N)$, effectively converting the hash table into a single linked list.
- **The Java Golden Rule:** *Always override* `equals()` *whenever you override* `hashCode()`, and vice versa.
- **Dynamic Resizing (Rehashing):** To maintain $O(1)$ operations, real-world implementations monitor load factor (λ) and dynamically rehash all entries into a larger array when capacity exceeds a given threshold.